

_s2_closer, in Pseudocode

every stage as an algorithm, every algorithm as a picture

Running example:

HI = the boy that chases the dog swims

HF = the the dog chases that boy swims

1 Notation: what the program manipulates

Everything is an **s-op**: a length- n array holding *one value per token position*. Positions are $0, \dots, n-1$. Nothing else exists — no stack, no tree, no recursion.

	<i>the</i>	<i>boy</i>	<i>that</i>	<i>chases</i>	<i>the</i>	<i>dog</i>	<i>swims</i>
	0	1	2	3	4	5	6
lc	D	N	THAT	V	D	N	V
role	0	0	0	2	0	0	1
dpfull	5	1	2	3	5	5	6

Each row is one s-op.
A *pointer* s-op (like dpfull) stores, at each position, an *index*.

1.1 The four primitives

Every line of the program is one of these. Only the last three move information between positions.

primitive	meaning	cost
MAP(x, f), WHERE(c, a, b)	elementwise	an MLP (free, no communication)
READ(x, δ)	$\text{out}[i] = x[i + \delta]$	1 attention head (relative offset)
GATHER(x, p)	$\text{out}[i] = x[p[i]]$	1 attention head (pointer chase)
STAB(ends)	$\#\{j \leq i : \text{ends}[j] \geq i\}$	1 counting head

The naming convention you must internalize

$A_at_B = \text{GATHER}(A, B)$, i.e. $A_at_B[i] = A[B[i]]$.

Read it as “*the value of A, fetched from the position named by B, delivered to i.*” The suffix names *where the value came from*. This is how a token reaches a distant constituent without ever using an absolute index: it computes *where* the thing it needs *starts*, then gathers that thing’s end-pointer from there.

Picture of Gather. Take $\text{dpfull_at_hb1} = \text{GATHER}(\text{dpfull}, \text{hb1})$ at the verb chases_3 . Its complement starts at $\text{hb1}[3] = 4$; we go read dpfull there and bring 5 back.

	0	1	2	3	4	5	6
hb1	1	2	3	4	5	6	7
dpfull	5	1	2	3	5	5	6
dpfull_at_hb1	1	2	3	5	5	6	0

Blue: $\text{hb1}[3] = 4$ says *where to look*. **Green:** $\text{dpfull}[4] = 5$ is the value carried back into $\text{dpfull_at_hb1}[3]$. So chases_3 learns “my complement starts at 4, and it ends at 5”.

2 Top level

HF is a *permutation* of HI, so the program never generates: it computes a target slot per token and gathers.

Algorithm 1 $\text{HiToHF}(w[0..n-1]) \text{ — head-initial} \rightarrow \text{head-final}$

Input: surface words w

Output: the head-final reordering

```

1: ids ← MAP( $w$ , WORD2ID)                                ▷ token embedding
2: i ← [0, 1, ...,  $n - 1$ ]                                ▷ position s-op
3: (lc, role, isThat, isCrel, isC) ← STAGE1(ids)            ▷ local, §3
4: (hbEnd, dpFlat, rcPos, hasRC) ← SCAFFOLD(...)           ▷ one-shot, §4
5: (dpfull, vpe, rce, cpe) ← RELAX(...)                   ▷ the loop, §5
6: e ← WHERE(isCrel, rce, WHERE(isC, cpe, i))              ▷ clause end per opener
7: depth ← STAB(WHERE(isThat, e, -1))                     ▷ nesting depth
8: disp ← d_adv + d_cp + d_nprel + d_vp                   ▷ §6
9: hfPos ← i + disp                                        ▷ target slot of each token
10: src ← INVERT(hfPos)                                    ▷ §7
11: return DECODE(GATHER(ids, src))

```

Only line 5 is hard. Lines 1–4 are local; lines 6–11 are counting and one gather.

3 Stage 1 — local part of speech and verb role

Algorithm 2 STAGE1 — everything from a token and its neighbours

```

1: lc ← MAP(ids, LEXCLASS)                                ▷  $\in \{D, N, Nprop, Adj, Adv, THAT, V\}$ 
2: prev, prev2 ← READ(lc, -1), READ(lc, -2)
3: next, next2 ← READ(lc, +1), READ(lc, +2)
4: isThat ← [lc = THAT]                                    ▷ the clause openers
5: isCrel ← isThat ∧ [prev = N]                            ▷ that after a common noun ⇒ relative
6: isC ← isThat ∧ ¬isCrel                                  ▷ otherwise sentential
7: nn ← WHERE([next = Adv], next2, next)                   ▷ next non-adverb class
8: if ¬[lc = V] then
9:   role ← 0                                              ▷ not a verb
10: else if [nn = THAT] then
11:   role ← 3                                              ▷ +clause (V_cp)
12: else if [nn ∈ {D, Nprop}] then
13:   role ← 2                                              ▷ +object DP (V_dp)
14: else if isIntrans then
15:   role ← 1                                              ▷ intransitive
16: else
17:   role ← 4                                              ▷ object-gap (transitive verb, no object)
18: end if

```

Why role is the whole point

Roles 2 and 3 are the verbs that **flip** (they have a complement to reverse with); roles 1 and 4 never flip. One forward neighbour read decides it.

On the running example (all values machine-verified):

	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
lc	D	N	THAT	V	D	N	V
isThat	0	0	1	0	0	0	0
isCrel	0	0	1	0	0	0	0
role	0	0	0	2	0	0	1

4 Stage 2a — one-shot scaffolding

Three fixed-offset facts, computed once, *before* any iteration.

Algorithm 3 SCAFFOLD — local span pieces

- 1: $\text{hbEnd} \leftarrow \text{WHERE}([lc=V] \wedge [next=Adv], i+1, i)$ ▷ verb (+adverb) head block
 - 2: $\text{dpFlat} \leftarrow \text{WHERE}([lc=D], \text{WHERE}([next=Adj], i+2, i+1), i)$ ▷ DP end *ignoring* a rel. clause
 - 3: $\text{rcPos} \leftarrow \text{dpFlat} + 1$ ▷ slot where a relative *that* would sit
 - 4: $\text{hasRC} \leftarrow [lc=D] \wedge [\text{GATHER}(lc, \text{rcPos}) = \text{THAT}]$ ▷ one lookahead
-

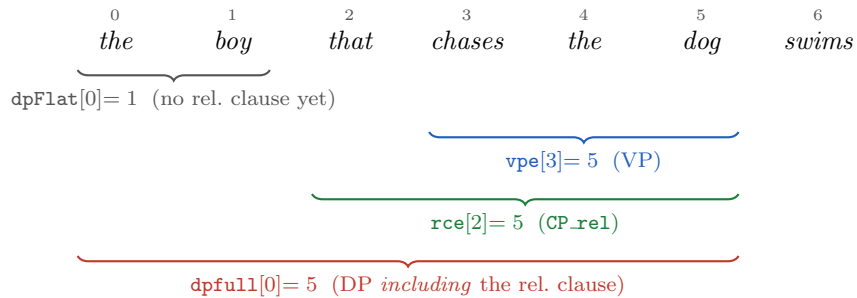
dpFlat by example. *the nice boy* $\Rightarrow \text{dpFlat} = [2, 1, 2]$: the determiner steps over the adjective ($i+2$); the other cells are the harmless i filler. *the boy* $\Rightarrow [1, 1]$.

hasRC by example. At *the*₀: $\text{rcPos}[0] = \text{dpFlat}[0] + 1 = 2$, and $\text{lc}[2] = \text{THAT} \Rightarrow \text{hasRC}[0] = 1$. That DP carries a relative clause.

5 Stage 2b — Relax: the heart

5.1 What we are trying to build

Four **end pointers**, each keyed by *start* position: $\text{ptr}[i] =$ last index of the constituent of that type that *begins* at i .



They are **mutually recursive** — each needs another’s answer:

$$\underbrace{\text{dpfull}}_{\text{DP}} \longrightarrow \underbrace{\text{rce}}_{\text{CP_rel}} \longrightarrow \underbrace{\text{vpe}}_{\text{VP}} \longrightarrow \underbrace{\text{cpe}}_{\text{CP_sent}} \longrightarrow \text{vpe} \longrightarrow \dots$$

so no single pass can compute them. We *relax*: seed a lower bound, re-apply the rules until nothing changes.

Algorithm 4 RELAX(K) — resolve the mutually recursive END pointers

```
1: dpfull  $\leftarrow$  dpFlat;  vpe  $\leftarrow$  hbEnd;  rce  $\leftarrow$  i;  cpe  $\leftarrow$  i ▷ seed: “each ends at itself”
2: for  $r = 1$  to  $K$  do ▷ one sweep = one attention hop = one nesting level
3:   dpfull  $\leftarrow$  WHERE(hasRC, GATHER(rce, rcPos), dpFlat) ▷ extend over the rel. clause
4:   ▷ a VP ends where its complement ends; the complement starts at hbEnd+1
5:   if role  $\in \{1, 4\}$  then
6:     vpe  $\leftarrow$  hbEnd ▷ no complement
7:   else if role = 2 then
8:     vpe  $\leftarrow$  GATHER(dpfull, hbEnd+1) ▷ object DP’s end
9:   else if role = 3 then
10:    vpe  $\leftarrow$  GATHER(cpe, hbEnd+1) ▷ embedded clause’s end
11:   end if
12:   ▷ a relative clause ends where its gap-clause ends
13:   if isCrel and [next = V] then
14:     rce  $\leftarrow$  GATHER(vpe, i+1) ▷ subject gap: it is a VP
15:   else if isCrel then
16:     rce  $\leftarrow$  GATHER(dpfull, i+1) + 1 ▷ object gap: the gap verb
17:   end if
18:   vpos  $\leftarrow$  GATHER(dpfull, i+1) + 1 ▷ embedded VP starts past the subject DP
19:   cpe  $\leftarrow$  WHERE(isC, GATHER(vpe, vpos), i)
20: end for
```

Loop invariant

After r sweeps, **every constituent with fewer than r levels of nesting beneath it has its exact end**. Each GATHER follows exactly *one* pointer, so one sweep advances the “resolved frontier” by exactly one level, from the inside out. Hence sweeps needed $\approx$ nesting depth, and ROUNDS is the program’s *depth budget*.

5.2 The ripple, depth 1

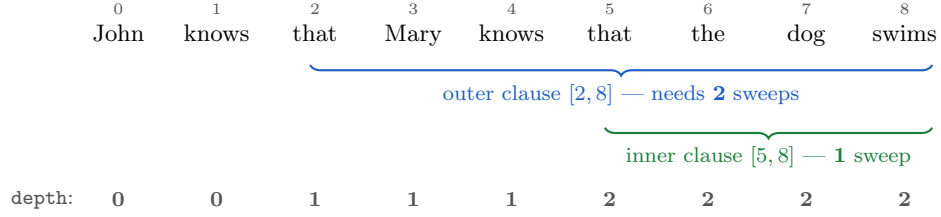
For our sentence the chain is $\text{dpFlat}[4] \rightarrow \text{vpe}[3] \rightarrow \text{rce}[2] \rightarrow \text{dpfull}[0]$. Because **dpfull** is updated *before* **rce** in each sweep, it lags one sweep behind:

	vpe[3]	rce[2]	dpfull[0]	
seed	3	2	1	“each ends at itself” / flat
sweep 1	5	5	2	dpfull read a <i>stale</i> rce
sweep 2	5	5	5	now correct
sweep 3–10	5	5	5	fixed point (nothing changes)

5.3 The ripple, depth 2 — why layers scale with depth

John knows that Mary knows that the dog swims: both clauses close on *swims*₈.

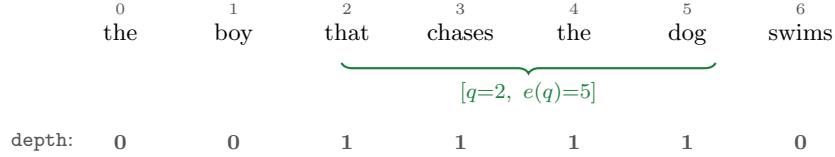
	cpe[5] (inner)	cpe[2] (outer)	
seed	5	2	
sweep 1	8	5	inner bottoms out at once; outer stops one level short
sweep 2	8	8	outer inherits the inner clause’s end



6 Stage 2c — depth, then displacement

6.1 Stab: turn intervals into a count

Each opener now owns an interval $[q, e(q)]$. $\text{depth}[i]$ counts how many cover i — one head.



6.2 Displacement = a sum of block swaps

A **flip** node lays its two children out as adjacent blocks and swaps them. A token in the **head** jumps right over the complement ($+|\text{comp}|$); a token in the **comp** jumps left over the head ($-|\text{head}|$). Nesting is *additive*.



`_s2_closer` groups that sum by **flip type** into four additive pieces. Each is a (weighted) STAB over the enclosing flips of that type, plus a local head bonus.

Algorithm 5 DISPLACEMENT — four additive families

- 1: $d_{\text{adv}} \leftarrow (+1 \text{ at a verb before an adverb}) + (-1 \text{ at an adverb})$ $\triangleright$ purely local swap
 - 2: $d_{\text{cp}} \leftarrow -\text{depth} + \text{WHERE}(\text{isThat}, e - i + 1, 0)$ $\triangleright$ clause body -1 each; the *that* $+|\text{clause}|$
 - 3: $d_{\text{nprel}} \leftarrow \underbrace{(+|\text{RC}| \text{ on the core noun})}_{\text{head bonus}} - \underbrace{\text{STAB}(\text{RC ends, weighted by } |\text{core}|)}_{\text{every token inside a RC}}$
 - 4: $d_{\text{vp}} \leftarrow \underbrace{(+|\text{comp}| \text{ on the verb block})}_{\text{head bonus}} - \underbrace{\text{STAB}(\text{comp ends, weighted by } |\text{head}|)}_{\text{every token inside a VP comp}}$
 - 5: **return** $d_{\text{adv}} + d_{\text{cp}} + d_{\text{nprel}} + d_{\text{vp}}$
-

Read one row of the example

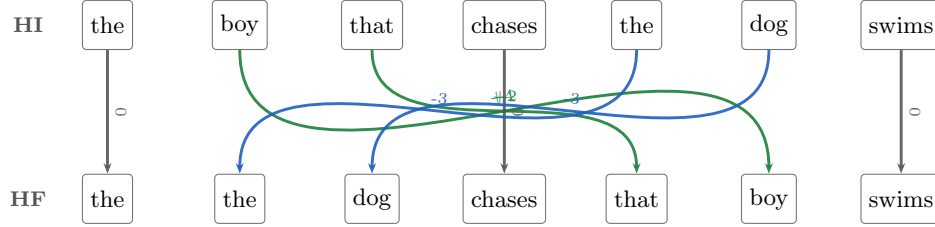
*chases*₃ is the **head** of its own VP ($+2$), but it also sits *inside* the relative clause (-1 from d_{cp}) and *inside* the NP's relative-clause complement (-1 from d_{nprel}). Total $+2-1-1=0$: it never moves.

7 Stage 3 — invert and gather

Algorithm 6 INVERT + gather — apply the permutation

- 1: $\text{hfPos} \leftarrow i + \text{disp}$ ▷ token i wants slot $\text{hfPos}[i]$
- 2: $\text{src}[j] \leftarrow \text{the unique } i \text{ with } \text{hfPos}[i] = j$ ▷ one head: match slot j against hfPos
- 3: **return** GATHER(ids, src) ▷ slot j takes the token at $\text{src}[j]$

Exactly one i matches each j because hfPos is a genuine bijection — that is the built-in correctness check.



8 End-to-end, every number

	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
lc	D	N	THAT	V	D	N	V
role	0	0	0	2	0	0	1
dpFlat	1	1	2	3	5	5	6
hasRC	1	0	0	0	0	0	0
dpfull	5	1	2	3	5	5	6
vpe	0	1	2	5	4	5	6
rce	0	1	5	3	4	5	6
e	0	1	5	3	4	5	6
depth	0	0	1	1	1	1	0
d_cp	0	0	+3	-1	-1	-1	0
d_nprel	0	+4	-1	-1	-1	-1	0
d_vp	0	0	0	+2	-1	-1	0
disp	0	+4	+2	0	-3	-3	0
hfPos	0	5	4	3	1	2	6
src	0	4	5	3	2	1	6

Reading **src**: slot 1 takes token 4 (*the*), slot 2 takes token 5 (*dog*), ... giving **the the dog chases that boy swims**.

The whole program in four sentences

- (1) Label each token from itself and two neighbours (**role** decides who flips).
- (2) Compute *where* each constituent's complement starts, then GATHER its end from there — repeating K times, because ends depend on nested ends.
- (3) Count enclosing intervals (STAB) to get **depth** and each token's $\pm$ shifts.
- (4) Add the shifts to get a target slot, invert, gather.

Every value above is machine-verified against `_s2_closer.hi_to_hf` and `grammar_oracle.py` (see `_closer_viz.py`).